

Avaliação de Branch Prediction em RISC-V com o simulador gem5: LocalBP versus BiModeBP e a sensibilidade ao padrão da carga

João Luís Almeida Santos¹

¹Bacharelado em Ciência da Computação
Organização de Computadores (ORG 2026.1)

joao.santos@abensoft.com.br

Abstract. *This work evaluates the impact of two conditional branch predictors, LocalBP (a simple local predictor) and BiModeBP (a complex global predictor), on a branch-heavy RISC-V workload: a Monte Carlo RPG combat engine running 1,000 automated matches per workload. The assembly code from the first assignment was ported to the GCC syntax, compiled to a static RISC-V binary and simulated on gem5 with an out-of-order CPU. We extract and compare misprediction rate, IPC, total simulated time and squashed instructions from the generated stats.txt. BiModeBP reduced the conditional misprediction rate from 11.27% to 8.57%, raised IPC by 3.8% and cut wasted speculative work by 38%.*

Resumo. *Este trabalho avalia o impacto de dois preditores de desvios condicionais, LocalBP (preditor local simples) e BiModeBP (preditor global complexo), sobre uma carga de trabalho com forte presença de desvios: um motor de combate de RPG por Monte Carlo que simula 1.000 partidas. O código assembly do primeiro trabalho foi adaptado para a sintaxe do GCC, compilado em um binário estático RISC-V e simulado no gem5 com uma CPU fora de ordem. Extraímos e comparamos a taxa de erro de predição, o IPC, o tempo total de simulação e as instruções descartadas do stats.txt gerado. O BiModeBP reduziu a taxa de erro de 11,27% para 8,57%, elevou o IPC em 3,8% e diminuiu em 38% o trabalho especulativo desperdiçado.*

1. Introdução

Preditores de desvio são componentes centrais de microarquiteturas modernas: em um pipeline profundo, cada desvio condicional mal previsto força o descarte (*squash*) de todas as instruções buscadas especulativamente após ele, pagando uma penalidade de vários ciclos [Hennessy and Patterson 2017]. O ganho de um bom preditor depende diretamente do padrão de desvios da carga de trabalho.

A carga avaliada é o algoritmo desenvolvido no Trabalho 1: um motor de combate de RPG escrito inteiramente em RISC-V que executa uma simulação Monte Carlo de 1.000 partidas entre três estratégias de IA distintas. Como cada turno é decidido por máquinas de estado e por números pseudoaleatórios (XORShift de Marsaglia [Marsaglia 2003]), o código é dominado por desvios condicionais dependentes de dados — o cenário ideal para estressar um preditor.

Seguindo o Tópico 2 da proposta (*Branch Prediction*), comparamos dois extremos de preditor condicional do gem5 [Binkert et al. 2011]: o **LocalBP** (local simples) e o **BiModeBP** (global complexo). As métricas exigidas — desvios avaliados e erros de predição — saem do `stats.txt` de cada execução, complementadas por IPC, tempo de simulação e instruções descartadas.

2. Fundamentação Teórica

2.1. LocalBP — preditor local

O **LocalBP** mantém uma tabela de contadores saturados de 2 bits indexada pelo PC do desvio; cada entrada guarda o histórico de tomadas/não-tomadas *daquele* desvio individualmente [Smith 1981]. É simples e barato, mas ignora correlação entre desvios distintos e sofre *aliasing* destrutivo quando vários desvios disputam a mesma entrada.

2.2. BiModeBP — preditor global bimodal

O **BiModeBP** [Lee et al. 1997] combina o histórico *global* recente com o PC para indexar duas tabelas de predição — uma “tomada” e outra “não tomada” — mais uma tabela de escolha. Ao separar os desvios por sua tendência predominante, reduz a interferência destrutiva entre desvios opostos e captura correlações globais, ao custo de mais área e complexidade.

3. Metodologia

3.1. Adaptação do código e compilação

O *assembly* original (`main.s`) foi escrito para o ambiente RARS, que oferece *syscalls* simplificadas e ponto de entrada próprio. Para o gem5 foi produzida a versão `main_gem5.s`, ajustando diretivas e o ponto de entrada `_start` para a sintaxe do GCC. O binário estático foi gerado com:

```
riscv64-linux-gnu-gcc -nostdlib -static main_gem5.s  
-o output/main_gem5
```

A `flag -static` produz o binário autocontido exigido pelo modo *Syscall Emulation* (SE) do gem5 e `-nostdlib` dispensa a *libc*. O artefato `output/main_gem5` é o binário simulado em ambos os cenários.

3.2. Configuração do gem5

O *script* de configuração (`models/inorder.py`) monta uma SimpleBoard a 1 GHz com hierarquia de cache privada L1 (32 kB I + 32 kB D) e L2 de 256 kB, e memória DDR3-1600 de 1 GB. Apenas o preditor condicional é trocado entre as execuções:

```
processor.cores[0].core.branchPred.conditionalBranchPred  
= LocalBP()  
# ou BiModeBP()
```

As duas simulações foram disparadas via `make run-bp`, gravando resultados em `m5out/local/` e `m5out/bimode/`.

Modelo de CPU. O MinorCPU (*In-Order* detalhado) sofre um *crash* conhecido na versão 25.1 do gem5 para RISC-V em modo SE (*bug* no `ActivityRecorder`); por isso

ambos os cenários usam o `DerivO3CPU`. Como o *único* parâmetro alterado entre as execuções é o preditor, a comparação isola o efeito da predição de desvios e ainda permite medir o trabalho especulativo descartado (Tópico 4).

Determinismo. A *seed* pseudoaleatória deriva do tempo de execução, logo os contadores absolutos diferem um pouco entre as execuções; a análise privilegia *taxas e razões* (taxa de erro, IPC), robustas a essa variação.

3.3. Duas cargas de trabalho

Como um preditor só ganha explorando *regularidade*, variamos o quão previsível é a carga — atendendo também ao pedido de “diferentes cargas de trabalho”. O motor de combate possui três bots: *Flowey* (RNG puro, sorteia a habilidade 1–6), *Chara* (combo determinístico) e *Toby* (contra-estratégia). Avaliamos duas cargas:

- **Mista** (`main_gem5.s`): a estratégia de cada jogador é sorteada por partida — mistura de bots determinísticos e aleatórios, parcialmente estruturada.
- **Aleatória** (`main_gem5_random.s`): ambos os jogadores são fixados em *Flowey*, de modo que toda decisão de habilidade é um sorteio uniforme. A variante foi *clonada* do original alterando apenas a atribuição de estratégia no `_start`, sem tocar no binário base.

Cada carga foi simulada com os dois preditores (4 execuções, 1.000 partidas cada).

4. Resultados

A Tabela 1 reúne as métricas extraídas do `stats.txt` de cada cenário. A taxa de erro de predição é calculada como `condIncorrect / condPredicted`.

Tabela 1. Métricas extraídas do `stats.txt` (`gem5`, `DerivO3CPU`).

Métrica	LocalBP	BiModeBP
Desvios cond. avaliados (<code>condPredicted</code>)	900.700	803.703
Desvios cond. incorretos (<code>condIncorrect</code>)	101.531	68.838
Taxa de erro de predição	11,27%	8,57%
IPC (<code>ipc</code>)	1,4603	1,5165
Tempo simulado (<code>simSeconds</code>)	7,539 ms	7,181 ms
Ciclos (<code>numCycles</code>)	7.538.818	7.180.697
Instruções efetivadas (<code>commit.numInsts</code>)	11.009.088	10.889.293
Instruções descartadas (<code>squashedInstsIssued</code>)	102.245	62.953
Acertos de BTB (<code>BTBHits/BTBLookups</code>)	71,9%	70,9%

4.1. Erros de predição

A Figura 1 mostra a métrica central do Tópico 2. O BiModeBP errou em 8,57% dos desvios condicionais contra 11,27% do LocalBP — uma redução relativa de **24%** na taxa de erro. A Figura 2 contextualiza: ainda que o número absoluto de desvios avaliados varie entre as execuções (efeito da *seed*), a fração de erros cai claramente com o preditor global.

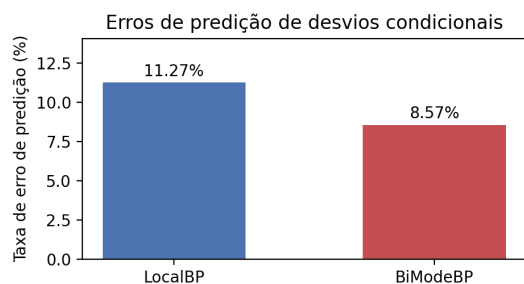


Figura 1. Taxa de erro de predição de desvios condicionais.

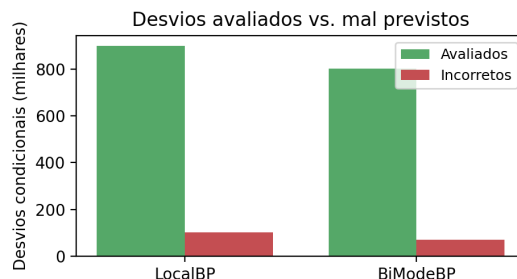


Figura 2. Desvios avaliados vs. mal previstos.

4.2. Impacto no desempenho

Menos erros de predição significam menos descartes e melhor aproveitamento do pipeline. O IPC subiu de 1,4603 para 1,5165 (+3,8%, Fig. 3) e o tempo de simulação caiu 4,7% (Fig. 4). O efeito mais marcante está na Figura 5: o BiModeBP descartou 62,9 mil instruções especulativas contra 102,2 mil do LocalBP — **38% menos trabalho desperdiçado**.

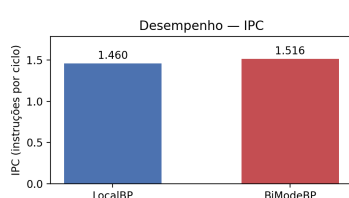


Figura 3. IPC.

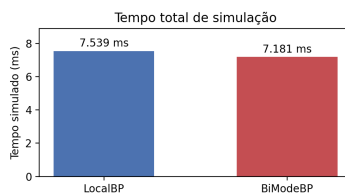


Figura 4. Tempo de simulação.

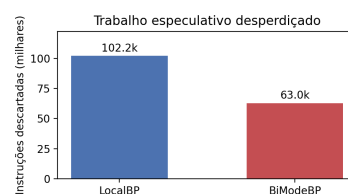


Figura 5. Descartadas.

4.3. Sensibilidade ao padrão de desvios da carga

A Tabela 2 repete o experimento na carga *aleatória* (Seção 3.3). Dois efeitos aparecem. Primeiro, **ambos** os preditores pioram: a taxa de erro do LocalBP sobe de 11,27% para 12,59% e a do BiModeBP de 8,57% para 10,07%, pois a escolha uniforme de habilidade transforma parte dos desvios em moedas justas, que nenhum histórico prevê. Segundo, e mais revelador, **a vantagem do BiModeBP encolhe**: de 24% para 20% de redução relativa do erro (Figura 7). Quanto mais aleatória a carga, menor o retorno do preditor sofisticado — seu ganho vem justamente da estrutura que a aleatoriedade destrói.

Tabela 2. Carga mista vs. aleatória, por preditor.

Métrica	Mista		Aleatória	
	Local	BiMode	Local	BiMode
Taxa de erro	11,27%	8,57%	12,59%	10,07%
IPC	1,460	1,517	1,418	1,480
Descartadas (mil)	102,2	62,9	167,0	126,5
Vantagem BiMode	−24% erro		−20% erro	

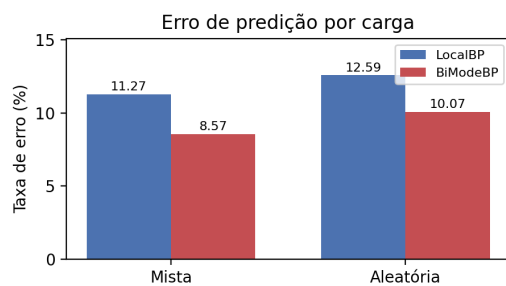


Figura 6. Taxa de erro por carga e preditor.

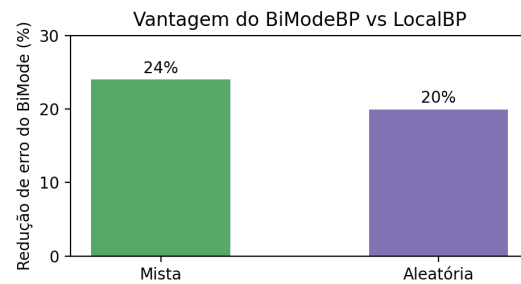


Figura 7. Vantagem relativa do BiModeBP encolhe na carga aleatória.

5. Discussão

Para esta carga de trabalho, o **BiModeBP é o preditor mais adequado**. O motor Monte Carlo é dominado por desvios dependentes de dados aleatórios e por máquinas de estado com muitos ramos correlacionados (ex.: “se $MP < 150$ então farmar; senão se HP do inimigo $> 50\%$ então atacar”). Essa correlação entre desvios é justamente o que o histórico global do BiModeBP captura e o LocalBP, restrito ao histórico por PC, não enxerga. A separação em tabelas “tomada” e “não tomada” também reduz o *aliasing* destrutivo gerado pela quantidade de pontos de decisão no laço das 1.000 partidas.

O ganho de desempenho (+3,8% de IPC), embora consistente, é moderado: parte dos desvios depende de valores genuinamente aleatórios e é, por natureza, imprevisível — nenhum preditor acerta uma moeda justa. O BiModeBP extrai o máximo da parcela *estruturada* (laços e máquinas de estado), ao custo de maior complexidade de *hardware*.

6. Conclusão

Comparamos LocalBP e BiModeBP sobre um binário RISC-V estático com forte carga de desvios, simulado no gem5. O preditor global complexo (BiModeBP) superou o local simples em todas as métricas: taxa de erro de 8,57% vs. 11,27%, IPC 3,8% maior, tempo 4,7% menor e 38% menos instruções descartadas. O resultado confirma a teoria: cargas com desvios correlacionados — máquinas de estado e laços de simulação — se beneficiam de preditores com histórico global, justificando seu custo de *hardware*.

O segundo experimento qualifica essa conclusão: ao tornar as decisões puramente aleatórias, os dois preditores pioram e a vantagem do BiModeBP cai de 24% para 20%. O valor de um preditor sofisticado não é absoluto — depende de quanta estrutura a carga oferece. Para este algoritmo, que mistura forte componente determinística com aleatoriedade, o BiModeBP ainda compensa, mas a margem é ditada pelo padrão de desvios da carga.

Referências

- Binkert, N., Beckmann, B., Black, G., Reinhardt, S. K., Saidi, A., Basu, A., Hestness, J., Hower, D. R., Krishna, T., Sardashti, S., Sen, R., Sewell, K., Shoaib, M., Vaish, N., Hill, M. D., and Wood, D. A. (2011). The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7.
- Hennessy, J. L. and Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition.

- Lee, C.-C., Chen, I.-C. K., and Mudge, T. N. (1997). The bi-mode branch predictor. In *Proceedings of the 30th Annual ACM/IEEE International Symposium on Microarchitecture (MICRO-30)*, pages 4–13.
- Marsaglia, G. (2003). Xorshift rngs.
- Smith, J. E. (1981). A study of branch prediction strategies. In *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA)*, pages 135–148.